THE WORKING SET MODEL FOR PROGRAM BEHAVIOR*

Peter J. Denning

Massachusetts Institute of Technology
Cambridge, Massachusetts

SUMMARY

Probably the most basic reason behind the absence of a general treatment of resource allocation in modern computer systems is an adequate model for program behavior. In this paper a new model is developed, the "working set model", which enables us to decide which information is in use by a running program and which is not. Such knowledge is vital for dynamic management of paged memories. The working set of pages associated with a process, defined to be the collection of its most recently used pages, is a useful allocation concept. A proposal for an easy-to-implement allocation policy is set forth; this policy is unique, inasmuch as it blends into one decision function the heretofore independent activities of process-scheduling and memory-management.

INTRODUCTION

Resource allocation is tricky business. In recent years there has been much dialogue on the topics of process scheduling and core memory management, yet development of techniques has progressed independently along both these lines. No one will deny that a unified approach is needed. Probably the most basic reason behind the absence of a general treatment is the lack of an adequate model for program behavior. In this paper we develop a new model, the working set model, which embodies certain important behavioral properties of computations operating in a multiprogrammed environment, enabling us to decide which information is in use by a running program and which is not. We do not intend that the proposed model be considered "final"; rather, we hope to stimulate a new kind of thinking, thinking that may be of considerable help in solving many operating system design problems.

The working set is intended to model the behavior of programs in the general purpose computer system, or computer utility. For this reason we assume that the operating system must on its own determine the behavior of programs it runs; it cannot count on outside help. Two commonly proposed sources of externally-supplied dynamic allocation information are the user and the compiler. We claim neither is adequate.

Because resources are multiplexed, each user is given the illusion that he has a complete computing system at his sole disposal: a virtual computer. For our purposes, the basic elements of a virtual computer are its virtual processor and an "infinite" one-level virtual memory. Dynamic "advice" regarding resource requirements cannot be obtained successfully from users for several reasons:

1. A user may build his program on the work of others, frequently sharing procedures whose time and storage requirements may be either unknown or, because of data dependence, indeterminate. Therefore he cannot be expected to estimate processor-memory needs.
2. It is not clear what sort of "advice" might be solicited. Nor is it clear how the operating system should use it, for overhead incurred by using advice could well negate any advantages attained.
3. Any advice acquired from a user would be intended (by him) to optimize the environment for his own program. Configuring resources to suit individuals may interfere with overall good service to the community of users.

Thus it seems inadvisable at the present time to permit users, at their discretion, to advise the operating system of their needs.

Likewise, compilers cannot be expected to supply information, extracted from the structure of the program**, regarding resource requirements:

1. Programs will be modular in construction; information about other modules may be unavailable at compilation time. Because of dependence on data there may be no way to decide (until run time) just which modules will be included in a computation.
2. Compilers cluttered with extra machinery to predict memory needs will be slower in operation. Many users are less interested in whether their programs operate efficiently than whether they operate at all, and so are concerned with rapid compilation. Furthermore, the compiler is an often-used component of the operating system; if slow and bulky, it can be a serious drain on system resources.

* Work reported herein was supported in part by Project MAC, an M.I.T. research project sponsored by the Advanced Projects Research Agency, Dept. of Defense, under Office of Naval Research Contract Number Nonr-4102(01).

** Ramamoorthy[1] has put forth a proposal for automatic segmentation of programs during compilation.

Therefore in this paper we are advocating mechanisms that monitor the behavior of a computation, making allocation decisions on the basis of currently observed characteristics. Only a mechanism that oversees the behavior of a program in operation can cope with arbitrary interconnections of arbitrary modules having arbitrary characteristics.

Our treatment proceeds as follows. First we define the type of computer system in which our ideas are developed. After a brief discussion of previous work with the problems of dynamic memory management, we define the working set model. We discuss a method of implementing memory management based on this model, and indicate how working set notions can be used to blend process scheduling and memory management into one decision function, accounting simultaneously for both types of demand. Finally we discuss how data sharing fits into the working-set scheme.

THE FRAMEWORK

We assume that the reader is already familiar with the concepts of a computer utility[2,3,4], of segmentation and paging[5,6], of program and addressing structure[6,8], so we will only mention these topics here. Briefly, each process has access to its own private, segmented name space; each segment known to the process is sliced into equal-size units, called pages, to facilitate mapping it into the paged main memory. Associated with each segment is a page table, whose entries point to the segment's pages. An "in-core" bit in each page table entry is turned ON whenever the designated page is present in main memory*; an attempt to reference a page whose "in-core" bit is OFF causes a page fault, initiating proceedings to secure the missing page. Finally, a process has three states of existence: running, when a processor is assigned to it; ready, when it would be running if only a processor were available; or blocked, when it has no need of a processor (for example, during a page fault or during a console interaction). When talking about processes in execution, we will have to distinguish between "process time" and "real time". Process time is time as seen by a process unaware it is suspended; that is, as if it executed without interruptions.

We restrict attention to a two-level memory system, indicated by Figure 1. Only data residing in main memory is accessible to a processor; all other data reside in auxiliary memory, which we regard as having infinite capacity. There is a time T, the traverse time, involved in transferring a page between memories. T is measured from the moment a page fault occurs until the moment the missing page is in main memory ready for use. T is actually the expectation of a random variable composed of waits in queues and mechanical positioning delays. Though it usually takes less time to store into auxiliary memory than to read from it, we shall regard the traverse time T to be the same regardless of which direction a page is moved.

*Consistent with current usage, we will use the terms "core memory" and "main memory" interchangeably.

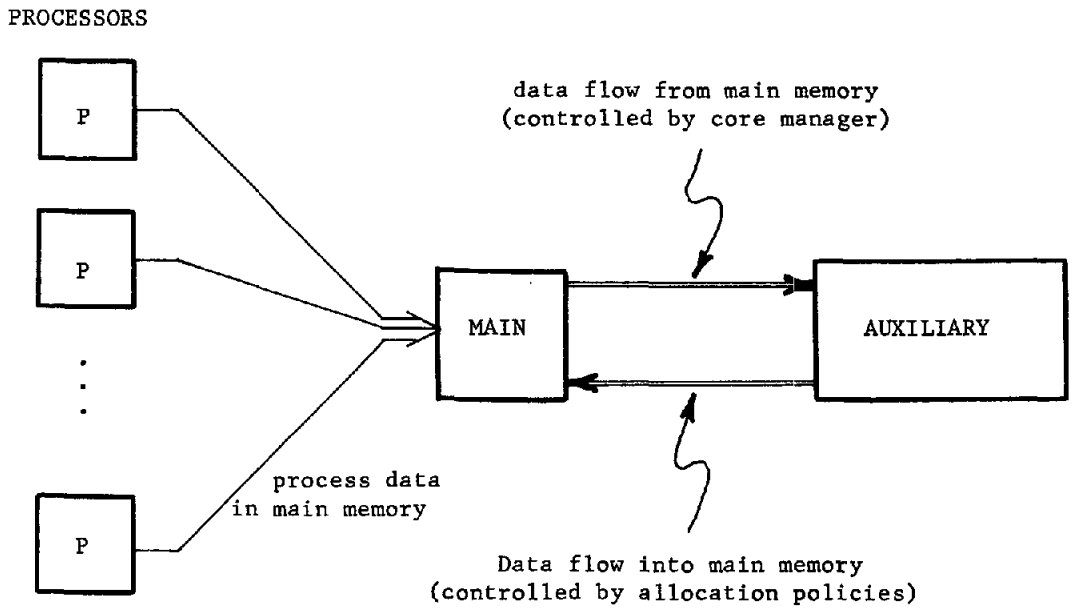

FIGURE 1. Two-level memory system.

A basic allocation problem, "core memory management", is that of deciding just which pages are to occupy main memory. The basic strategy advocated here -- a compromise against a lot of expensive main memory -- is to minimize page traffic*. There are two reasons for this:

1. The more the data traffic between the two levels of memory, the more the computational overhead in deciding just what to move and where to move it.
2. Because the traverse time T is long compared to a memory cycle, too much data movement can result in congestion and serious interference with processor efficiency.

Roughly speaking, a working set of pages is the minimum collection of pages that must be loaded in main memory for a process to operate efficiently, without "unnecessary" page faults. According to our definitions, a "process" and its "working set" are but two manifestations of the same ongoing computational activity.

PREVIOUS WORK

In this section we outline strategies that have been set forth in the past for memory management; the interested reader will be referred to the literature for detail.

We regard management of paged memories to operate in two stages:

1. Paging in: locate the required page in auxiliary memory, load it into main memory, turn the "in-core" bit of the appropriate page table entry ON.
2. Paging out: remove some page from main memory, turn the "in-core" bit of the appropriate page table entry OFF.

Management algorithms can be classified according to their methods of paging in and paging out. It is a common characteristic of nearly every strategy that paging in is done on demand; that is, no action is taken to load a page into memory until some process attempts to reference it. To date there have been no proposals recommending look-ahead, or anticipatory page-loading, because (as we have stressed) there is no reliable advance source of allocation information, be it the programmer or the compiler. Although the working set is the desired information, it might still be futile to pre-load pages: there is no guarantee that a process will not block shortly after resumption, having referenced only a fraction of its working set. The operating system could devote its already precious time to activities more rewarding than loading pages which may not be used. Thus we will assume that paging in is done on demand only, via the page fault mechansim.

The chief problem in memory management is not deciding which pages to load; it is deciding which pages ought to be removed. For if the page with the least likelihood of being used in the immediate future is retired to auxiliary memory, the best choice has been made. Nearly every worker in the field has recognized this. Debate has arisen over which strategy to employ for retiring pages; that is, which page-turning, or replacement, algorithm to use.

A good measure of performance for a paging policy is page traffic (the number of pages per unit time being moved between memories), since erroneously removed pages add to the traffic of returning pages. In the following we will use this as a basis of comparison for several strategies.

Random selection. Whenever a fresh page of memory is needed, a page is selected at random to be replaced. Although utterly simple to implement, this method frequently removes useful pages (which must therefore be recalled) and so results in high page traffic.

Cyclic selection. The pages of main memory are ordered in a cyclic list. Suppose the M pages of main memory are numbered $0,1,\ldots,(M-1)$ and a pointer k indicates that the k-th page was most recently paged in. Whenever a fresh page of memory is needed, $[(k+1) \bmod M] \rightarrow k$, page k is retired, and another page brought in to fill the now vacant slot. This method -- also utterly simple to realize -- is based on the principle that programs tend to follow sequences of instructions, so that references in the immediate future will most likely be close to present references. So, assuming there is this tendency for page references to cluster, and assuming some kind of uniformity in process scheduling techniques, the page which has been in memory longest is least likely to be reused: hence the cyclic list. We see two ways in which this algorithm can fail. First we question its basic assumption. It is not at all clear that modular programs, which execute numerous inter-module calls, will indeed exhibit sequential instruction fetch patterns. The thread of control will not string pages together; rather, it will entwine them intricately. [Fine, McIssac, and Jackson[9] have some experimental evidence in support of this reasoning.] Second, this algorithm is subject to overloading when used in multiprogrammed memories. When core demand is too heavy, one cycle of the list completes rapidly and the pages deleted are still needed by their processes. This can create a self-intensifying crisis. Programs, deprived of still-needed pages, generate a plethora of page faults; the resulting traffic of returning pages displaces still other useful pages, leading to more page faults, and so on.

Oldest-unused selection. Each page table entry contains a "use" bit, set ON each time the page is referenced. At periodic intervals all the page table entries are searched and usage records updated. When a fresh page of memory is needed, the page unreferenced for the longest time is removed. One can

*Since data is stored and transmitted in units of pages, we can (without ambiguity) refer to data movement as "page traffic".

see that this method is intrinsically reasonable by considering the simple case of a computer where there is exactly one process whose pages cannot all fit into main memory. In this case the most reasonable choice for a page to replace is the oldest unused page. Unfortunately this method too is susceptible to overloading when many processes compete for main memory.

ATLAS loop detection method. The Ferranti ATLAS computer[10] had proposed a page-turning policy that attempted to detect loop behavior in page reference patterns, then minimize page traffic by maximizing the time between page transfers, that is, by removing pages not expected to be needed for the longest time. It was successful -- only for looping programs. Performance was unimpressive for programs exhibiting random reference patterns. Implementation was costly.

Various studies concerning behavior of paging algorithms have appeared. Fine, McIssac and Jackson[9] have investigated the effects of demand paging and have questioned whether paging is beneficial at all. We do not feel that their conclusion applies to the kind of multiprogrammed environment we have described. They studied fixed-size programs, that quickly acquired and retained a large fraction of their pages. Highly interactive, modular programs are likely to behave differently. Not only may program size vary dynamically (according to data dependencies), but also such programs should be using a small fraction of their pages at any one time, and the membership in this set of working pages should be changing constantly.

Belady[11] has compared some of the algorithms mathematically. His most important conclusion is that the "ideal" algorithm should possess much of the simplicity of Random or Cyclic selection (for efficiency) and some, though not much, accumulation of data on past reference patterns. He has shown that too much "historical" data can have adverse effects (witness ATLAS).

In the next section we begin investigation of the working set concept. Even though the ideas are not entirely new[12,13,14], there has been no detailed documentation publicly available.

THE WORKING SET MODEL

From the programmer's standpoint, the working set of information is the smallest collection of procedure and data items that must be present in main memory to assure efficient execution of his program. We have already stressed that there will be no advance notice from either the programmer or the compiler regarding what information "ought" to be in main memory. It is up to the operating system to determine on the basis of page reference patterns whether pages are in use. Therefore the working set of information associated with a process is, from the system standpoint, the set of most recently referenced pages.

We define the working set of information $W(t,\tau)$ of a process at time t to be the collection of data items referenced by the process during the process time interval $(t-\tau,t)$.

Thus, the data items a process has referenced during the last τ seconds of its execution comprise its working set. τ will be called the working set parameter. We will regard the data items in $W(t,\tau)$ as being pages, although they could just as well be any other named data objects. The working set size $\omega(t,\tau)$ is

(1) $\omega(t,\tau)$ = Number of pages in $W(t,\tau)$

A working set $W(t,\tau)$ has two important, general properties. Both are properties of typical programs, and need not hold in special cases.

P1. Size. It should be clear immediately that $\omega(t,0) = 0$ since no page reference can occur in zero time. It should also be clear that $\omega(t,\tau)$ as a function of τ is monotonically increasing, since more pages can be referenced in longer time intervals. Because a process will refer to its most-needed pages frequently and its least-needed pages infrequently, we expect $\omega(t,\tau)$ as a function of τ to have a steep initial rise which diminishes to a more gradual rise. The general character of $\omega(t,\tau)$ is suggested by the smoothed curve of Figure 2.

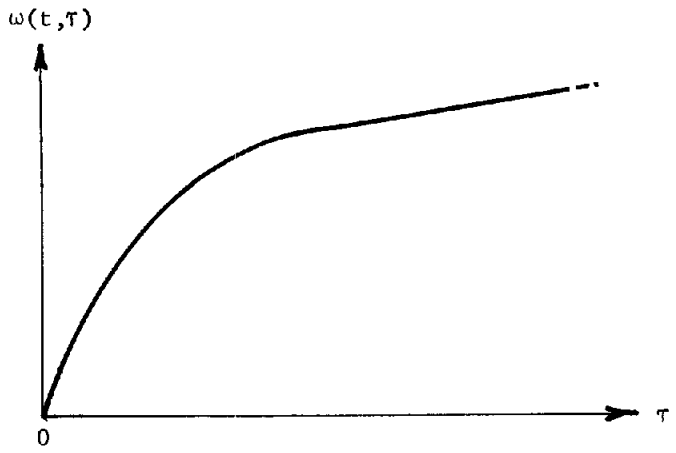

FIGURE 2. Behavior of $\omega(t,\tau)$.

P2. Correlation. Program modularity enables us to say something about correlation between the size of a working set at two times, t and $(t+\alpha)$. Correlation is useful in devising storage allocators, for the higher the correlation between $\omega(t,\tau)$ and $\omega(t+\alpha,\tau)$, the better is $\omega(t,\tau)$ a prediction of $\omega(t+\alpha,\tau)$. In modular programs, control passes randomly from one module to another; if τ is chosen properly (as discussed in the next section), it is more likely that a working set will change size smoothly, less likely that it will change size abruptly. Thus for small time separations α (say, $\alpha \ll \tau$), $\omega(t,\tau)$ and $\omega(t+\alpha,\tau)$ are highly

correlated, meaning that a measurement of $\omega(t,\tau)$ will serve as a good estimate of the memory requirement during the process time interval $(t,t+\alpha)$. For large time separations α (say, $\alpha \gg \tau$), control will have passed through a great many modules during the interval $(t,t+\alpha)$; thus $\omega(t,\tau)$ gives little information about $\omega(t+\alpha,\tau)$, and so $\omega(t,\tau)$ and $\omega(t+\alpha,\tau)$ have much less correlation than for small α. This behavior is suggested in Figure 3.

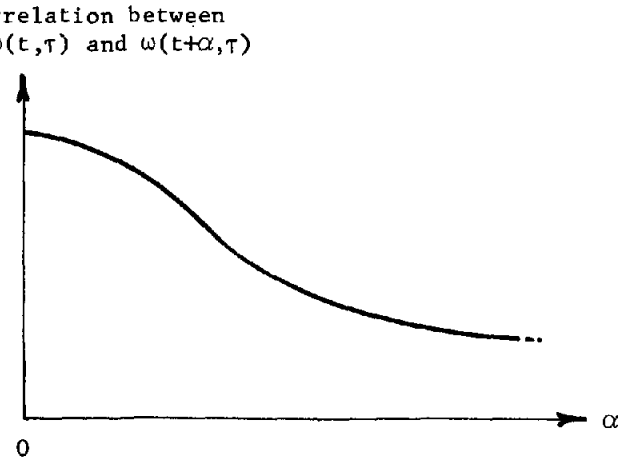

FIGURE 3. Correlation between working set sizes.

Choice of τ

The value ultimately selected for τ will reflect efficiency requirements and will be influenced by system parameters such as core memory size and memory traverse time. For example, if τ is too small, pages may be removed from main memory while they are still useful, and high page traffic may result from returning pages. If τ is too large, pages may remain in main memory long after last being used, and wasted main memory may result. Thus the value of τ will have to represent a compromise between too much page traffic and too much wasted memory space.

The following consideration leads us to recommend for τ a value comparable to the memory traverse time T (Figure 1). Consider a process that is running continuously, being interrupted only for page faults. Assuming that memory allocation procedures balk at removing from main memory any page in a working set, once a page has entered $W(t,\tau)$ it will remain in main memory for at least τ seconds. Under the very worst of page-shuffling conditions, a page could be dispatched to auxiliary memory and be recalled immediately; the time for this round trip is two traverse times, 2T. Therefore a highly-shuffled page would spend roughly $\tau/2T$ of its time in main memory. So, for example, if we wished to insure that a page is available in main memory (when needed) for not less than 50 per cent of the time, we would have to choose $\tau \approx 2T$.

Detecting $W(t,\tau)$

According to our definition, $W(t,\tau)$ is the set of its pages a process has referenced within the last τ seconds of its execution. This suggests that memory management can be controlled by hardware mechanisms, by associating with each page of main memory a timer. Each time a page is referenced, its timer is set to τ and begins to run down; if the timer succeeds in running down, a flag is set to mark the page for removal whenever the space is needed. In the appendix we describe such a hardware memory management mechanism, hardware that can be housed within the memory boxes. The mechanism has two interesting features:

1. It operates asynchronously and independently of the supervisor, whose only respsonsibility in memory management is handling page faults. Quite literally, memory manages itself.
2. Analog devices such as capacitative timers could be used to measure intervals.

Unfortunately it is not practical to add on hardware to existing systems. We seek a method of handling memory management within the software. The procedure we propose here samples the page table entries of pages in core memory at process time intervals of σ seconds (σ is called the sampling interval) where $\sigma = \tau/K$, K an integer constant chosen to make the sampling intervals as "fine grain" as desired. On the basis of page references during each of the last K sampling intervals, the working set $W(t,K\sigma)$ can be determined, as follows.

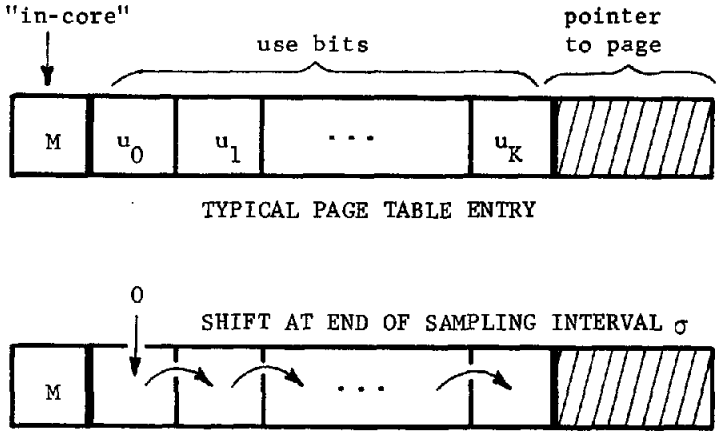

FIGURE 4. Page table entries for detecting $W(t,K\sigma)$.

As indicated by Figure 4, each page table entry contains an "in-core" bit M, where M=1 if and only if the page is present in main memory. It also contains a string of use bits $u_0,u_1,\ldots,u_K$. Each time a page reference occurs, $1 \rightarrow u_0$. At the end of each sampling interval σ, the bit pattern contained in $u_0,u_1,\ldots,u_K$ is shifted one position, a 0 enters u_0, and u_K is discarded:

$$
\begin{array}{rcl}
u_{K-1} & \rightarrow & u_K \\
 & \vdots & \\
u_0 & \rightarrow & u_1 \\
0 & \rightarrow & u_0
\end{array}
\qquad (2)
$$

Then the logical sum U of the use bits is computed:

$$U = u_0 + u_1 + \ldots + u_K \qquad (3)$$

so that U=1 if and only if the page has been referenced during the last K sampling intervals; of all the pages associated with a process, those with U=1 comprise its working set $W(t,K\sigma)$. If U=0 when M=1 the page is no longer in a working set and may be removed from main memory.

Memory Allocation

In our discussion so far we have seen two alternative quantities of possible use in storage allocation: the working set $W(t,\tau)$ and the working set size $\omega(t,\tau)$. We advocate use of $\omega(t,\tau)$.

Complete knowledge of $W(t,\tau)$, page for page, would be needed if look-ahead were contemplated. We have already discussed why past paging policies have shunned look-ahead, due to the strong possibility that pre-loading could be futile. A program organization likely to be typical of interactive, modular programs, shown in Figure 5, fortifies our previous argument against look-ahead. The user sends requests to the interface procedure A; having interpreted the request, A calls on one of the procedures $B_1,\ldots,B_n$ to perform an operation on the data D. The called B-procedure then returns to A for the next user request. Each time the process of this program blocks, the working set $W(t,\tau)$ is likely to change radically -- sometimes only A may be in $W(t,\tau)$, at other times one of the B-procedures and D may be in $W(t,\tau)$. The pages of $W(t,\tau)$ are likely to be different after blocking for an interaction from before blocking. Thus, the fact that a process blocks for an interaction (not page faults) can be a strong indication of a change in $W(t,\tau)$. Therefore the look-ahead, most often used just after a process unblocks, would probably load pages not likely to be used.

Knowledge of only $\omega(t,\tau)$ with demand paging suffices to manage memory well. Before running a process we insure that there are enough pages of memory free to contain its working set $W(t,\tau)$, whose pages fill free slots upon demand. By implication, enough free storage has been reserved so that no page of a working set of another running process is displaced by a page of $W(t,\tau)$ (as can be the case with the Random, Cyclic, or Oldest-unused paging policies). Accordingly we will use the working set size $\omega(t,\tau)$ as a measure of memory demand for storage allocation.

SOFTWARE IMPLEMENTATION

The previous discussion has indicated a skeleton for implementing memory management using working sets. Now we will fill in the flesh.

If the working set ideas are to contribute to good service, an implementation should have these properties:

1. Since there is such an intimate relation between a process and its working set, memory management and process scheduling must be closely related activities. One cannot take place independently of the other.
2. Efficiency should be of prime importance. When sampling of page tables is done, it

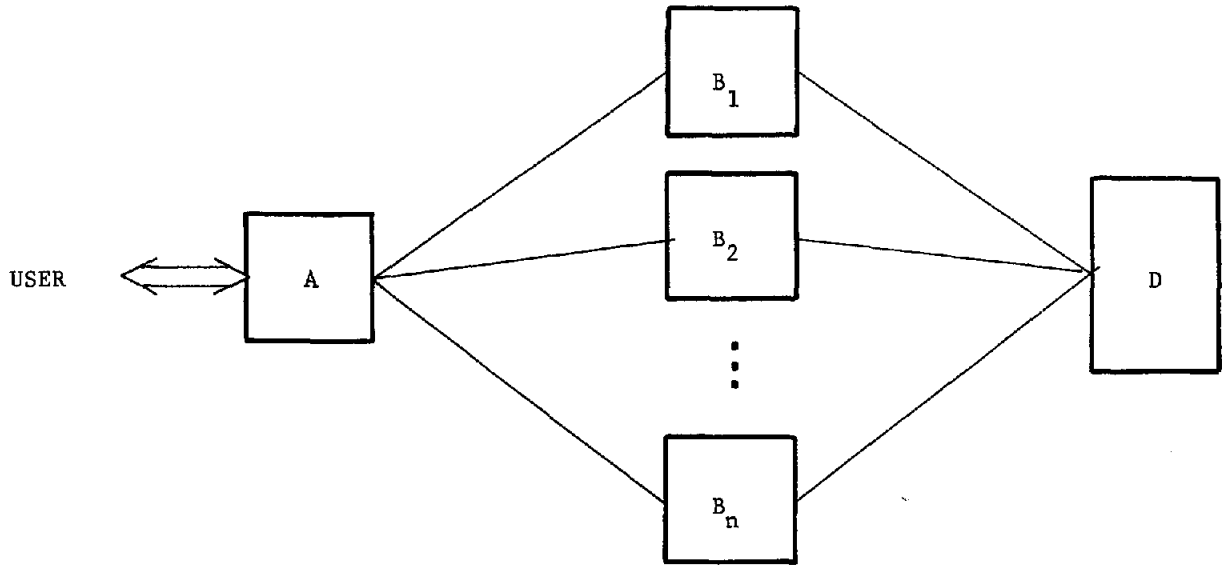

FIGURE 5. Organization of a program.

should be only on pages in currently changing working sets, and it should be done as infrequently as possible.

3. The mechanism ought to be capable of providing measurements of current working set sizes and processor time consumptions for each process.

Figure 6 displays an implementation having the desired properties. Each solid box represents a delay. The solid arrows indicate the paths that may be followed by a process identifier while it traverses the network of queues. The dashed boxes and arrows show when operations are to be performed on the time-used variable t_i associated with process i; processor time used by process i since it was last blocked (page faults excluded) is recorded in t_i. We shall follow a single process through this system to see what transpires:

1. When process i is created, an identifier for it is placed in the ready list, which is a list of all processes in the ready state, demanding service. Processes are selected from the ready list to enter service according to the prevailing priority rule.

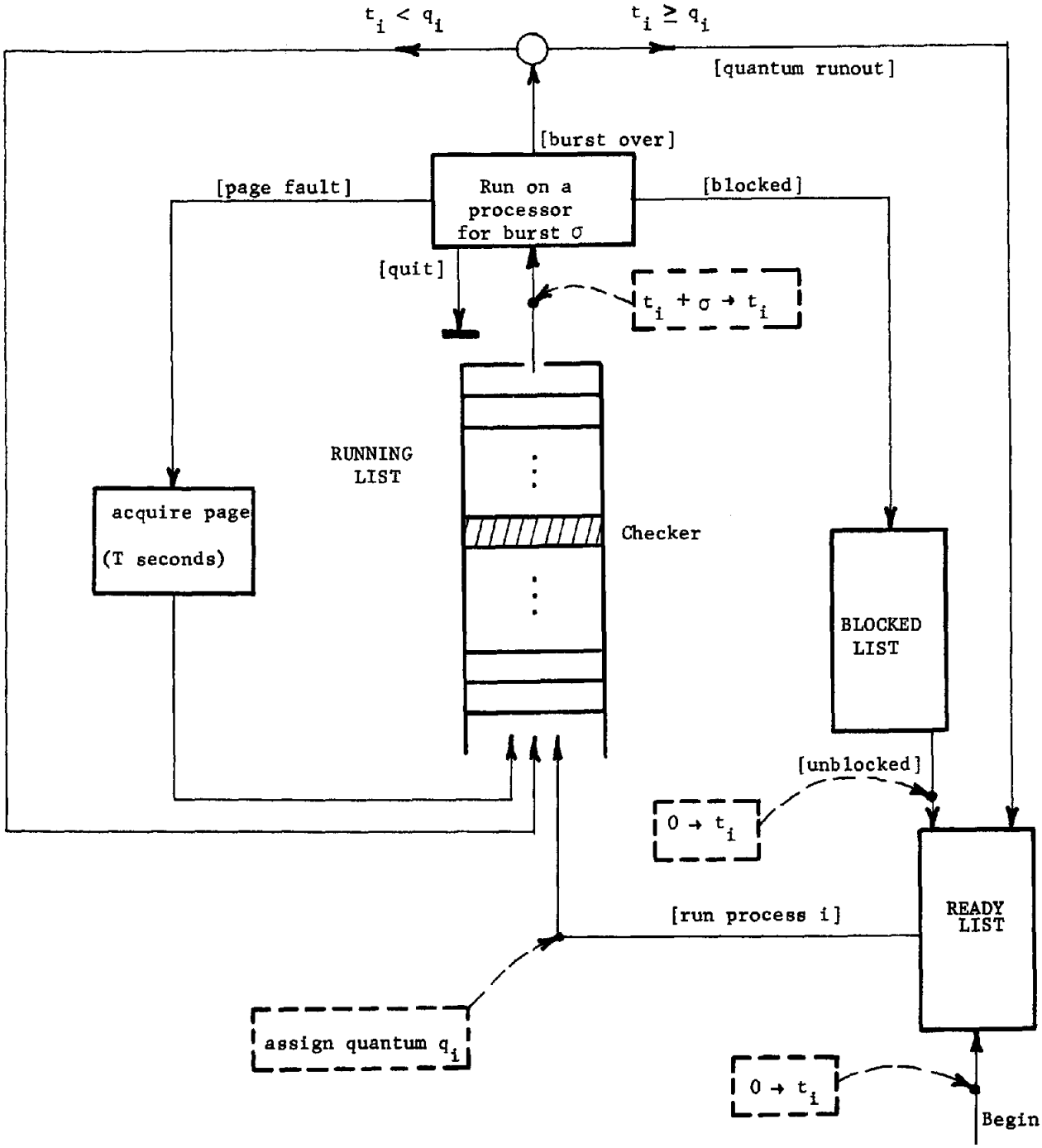

FIGURE 6. Implementation of Scheduling.

2. Once selected from the ready list, process i is assigned a qunatum q_i, which upper-bounds its time in the running list. This list is a cyclic queue; process i cycles through repeatedly, receiving bursts σ of processor time until it blocks or exceeds its quantum q_i. Note that the processor burst σ is also the sampling interval.
3. If process i blocks, its identifier is placed in the blocked list, where it remains until the process unblocks; it is then re-entered in the ready list.

Perennially present in the running list is a special process, the checker. The checker performs core management functions. It samples the page tables of each process that has received service since the last time it (the checker) was run, removing pages according to the algorithm discussed at equations 2 and 3. It should be clear that if the length of the running list is ℓ, sampling of page tables occurs every $\ell\sigma$ seconds, not every σ seconds.

Associated with process i is a counter w_i giving the current size $\omega_i(t,\tau)$ of its working set. Each time a page fault occurs a new page enters $W_i(t,\tau)$ and so w_i must be increased by one. Each time a page is removed from $W_i(t,\tau)$ by the checker, w_i is decreased by one.

Having completed its management duties, the checker replenishes vacancies in the running list by selecting jobs from the ready list according to the prevailing priority rule. More will be said about this shortly.

SHARING

Sharing finds its place naturally.

When pages are shared, working sets will overlap. If Arden's[7] suggestion concerning program structure* is followed, sharing of data can be accomplished without modification of the regime of Figure 6. If a page is in at least one working set, the "use bits" in the page table entry will be turned on and the page will not be removed. To prevent anomalies, the checker must not be permitted to examine the same page table more than once during one of its samples. Allocation policies should tend to run two processes together in time whenever they are sharing information (symptomized by overlap of their working sets) in order to avoid unnecessary reloading of the same information. How processes should be charged for memory usage when their working sets overlap is still an open question, and is under investigation.

*If a segment is shared, there will be an entry for it in the segment tables of each participating process; however, each entry points to the same page table. Each physical segment has exactly one page table describing it, but a name for the segment may appear in many segment tables.

USE OF WORKING SETS IN RESOURCE ALLOCATION

In this section we want to indicate how information about working set size and processor time consumption can be used by allocation procedures to select the next job from the ready list. In Figure 6 we have tried to indicate that the interactions between memory and processor requirements cannot be ignored. Here, we shall propose an allocation policy that incorporates both process scheduling and memory management into one function. We begin by defining carefully the notion "demand" and showing how it can be measured dynamically by the operating system. From "demand" we proceed to "balance". The objective of the allocation policy will be to keep the computer system "balanced". We have chosen this as an objective function primarily because of its mathematical simplicity.

Demand

Our purpose here is to define "memory demand" and "processor demand", then combine these into the single notion "demand".

We define the memory demand m_i of process i to be

$$(4) \qquad m_i = \min\left(\frac{w_i}{M}, 1\right) \qquad 0 \le m_i \le 1$$

where M is the number of pages of main memory, and $w_i = \omega_i(t,\tau)$ is the working set count, such as maintained by the strategy of Figure 6. If a working set contains more than M pages (it is bigger than main memory) we regard its demand to be m=1. Presumably M is large enough so that the probability (over the ensemble of all processes) Pr[m=1] is very small.

"Processor demand" is difficult to define without some discussion. Just as memory demand is in some sense a prediction of memory requirements in the immediate future, so too processor demand should be a prediction of processor requirements for the near future. There are many ways in which processor demand could be defined. The method we have chosen, described below, defines the processor demand of a process to be its expected fractional processor requirement before the next time it blocks (exclusive of page faults).

Let q be the random variable of processor time used by a process between interactions. [A process "interacts" when it communicates with something outside its name space, e.g., with a user, or with another process.] In general character, $f_q(x)$, the probability density function for q, is hyper-exponential (for a complete discussion, see Fife[15]):

$$(5) \qquad f_q(x) = c\,a\,e^{-ax} + (1-c)\,b\,e^{-bx} \qquad \begin{matrix} 0<a<b \\ 0<c<1 \end{matrix}$$

$f_q(x)$ is diagrammed in Figure 7; most of the probability is concentrated toward small q (i.e., frequently interacting processes), but $f_q(x)$ has a long exponential tail.

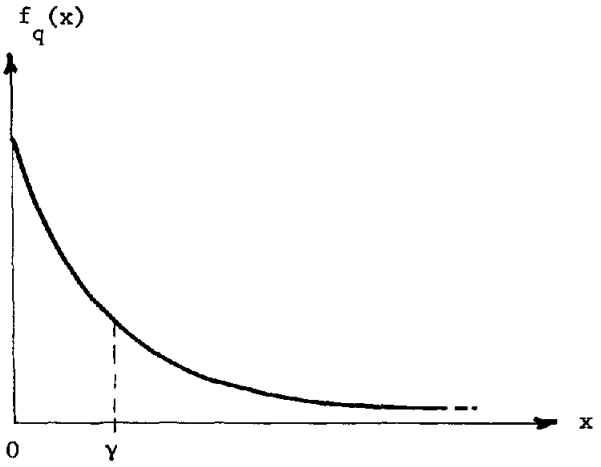

FIGURE 7. Probability density function for q.

Given that it has been γ seconds (process time) since the last interaction, the conditional density function for time until next interaction is

$$(6) \qquad f_{q|\gamma}(x) = \frac{f_q(x+\gamma)}{\int_\gamma^\infty f_q(z)\,dz} = \frac{c\,a\,e^{-a(x+\gamma)} + (1-c)\,b\,e^{-b(x+\gamma)}}{c\,e^{-a\gamma} + (1-c)\,e^{-b\gamma}}$$

which is just that portion of $f_q(x)$ for $q \geq \gamma$ with its area normalized to unity. The conditional expectation of q, given γ, is:

$$(7) \qquad Q(\gamma) = \int_0^\infty x\, f_{q|\gamma}(x)\,dx = \frac{\frac{c}{a}\,e^{-a\gamma} + \frac{(1-c)}{b}\,e^{-b\gamma}}{c\,e^{-a\gamma} + (1-c)\,e^{-b\gamma}}$$

The conditional expectation function $Q(\gamma)$ is shown in Figure 8.

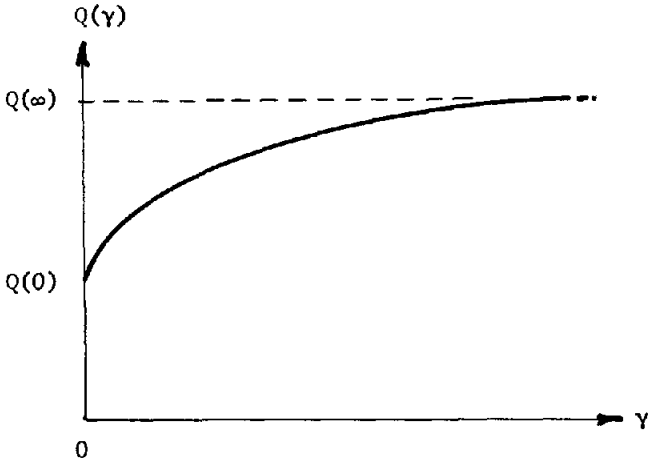

FIGURE 8. Conditional expectation function for q.

It starts at $Q(0) = \frac{c}{a} + \frac{1-c}{b}$ and rises toward a constant maximum of $Q(\infty) = \frac{1}{a}$. Note that, for large enough γ, the conditional expectation becomes independent of γ. The conditional expectation $Q(\gamma)$ is a useful prediction function -- if γ seconds of processor time have been consumed by a process since its last interaction, we may expect $Q(\gamma)$ seconds of process time to elapse before its next interaction. It should be clear that the conditional expectation function $Q(\gamma)$ can be determined and updated automatically by the operating system.

We define the processor demand p_i of process i to be

$$(8) \qquad p_i = \frac{Q(t_i)}{N\,Q(\infty)}, \qquad \frac{Q(0)}{N\,Q(\infty)} \leq p_i \leq \frac{1}{N}$$

where N is the number of processors and t_i is the time-used quantity for process i, such as maintained by the strategy of Figure 6.

The demand $\underline{D}_i$ of process i is a pair

$$(9) \qquad \underline{D}_i = (p_i, m_i)$$

where p_i is its processor demand (equation 8) and m_i is its memory demand (equation 4). That the processor demand is p_i tells us to expect process i to use p_i of the processors for the next $Q(\infty)$ seconds, before its next interaction*. That the

*A reasonable choice for the quantum q_i (Figure 6) granted to process i might be $q_i = k\,Q(t_i)$ for some suitable constant $k \geq 1$.

memory demand is m_i tells us process i is most probably going to use ($m_i M$) pages of memory during the next few time units.

Balance

The computer system is said to be balanced if simultaneously

$$(10) \qquad \sum_{\substack{\text{processes in} \\ \text{running list}}} p = \alpha \qquad 0 < \alpha \leq 1$$

$$(11) \qquad \sum_{\substack{\text{processes in} \\ \text{running list}}} m = \beta \qquad 0 < \beta \leq 1$$

where p is a processor demand, m a memory demand, and α,β are constants chosen to cause any desired fraction of resource to constitute balance. If the system is balanced, the total demand presented by the running list processes just consumes the available fractions of processor and memory resources. We can write equations 10 and 11 in the more compact form

$$(12) \qquad \underline{S} = \sum_{\substack{\text{processes in} \\ \text{running list}}} \underline{D} = (\alpha,\beta) \qquad \underline{D} = (p,m)$$

so that balance exists whenever equation 12 holds, that is, whenever $\underline{S} = (\alpha,\beta)$.

Dynamic maintenance of $\underline{S} = \Sigma \underline{D}$ is straightforward. Whenever a process of demand (p,m) is admitted to the running list, $\underline{S} + (p,m) \rightarrow \underline{S}$. Whenever a process of demand (p,m) exits the running list (except for page faults), $\underline{S} - (p,m) \rightarrow \underline{S}$. Therefore $\underline{S}$ always measures the current total running list demand.

Balance Policies

A "balance policy" is a resource allocation policy whose objective is to keep the computer system in balance. Expressed as a minimization problem:

$$(13) \qquad \{\text{minimize } |\underline{S} - (\alpha,\beta)| \}$$

Instead of a priority in the ready list, a process has associated its demand $\underline{D}$. In the event of imbalance, the next job (or set of jobs) to leave the ready list should be that whose demand comes closest to restoring balance. [Means of formulating this type of policy are currently under investigation as part of doctoral research into the whole problem of resource allocation.]

We do not wish to venture further here into the alluring problems of allocation policies; our aim is primarily to stimulate new thinking by presenting seeds of ideas. There are, however, two points we want to stress about policy (13):

1. The criterion is basically an equipment utilization criterion. It is well known that equipment utilization and good response to users are not mutually-aiding criteria. As it stands, policy (13) will favor jobs of small demand, discriminate against jobs of large demand; but with modifications and the methods suggested by Figure 6, it is possible to maintain "almost-balance" along with good service to users.
2. Even with intuitively simple strategies such as balance, the allocation problem is far from trivial -- interactions such as those between process and working set, and between balance and good service, are not yet fully understood.

CONCLUSION

In a synopsis of previous work on core memory management culled from varied sources we saw that memory management operates in two basic stages: page-in and page-out. Page-in should be done on demand, without look-ahead; page-out can be done in a variety of ways. Page-out is the heart of the problem, for if pages least likely to be reused in the near future are removed from main memory, the traffic of returning pages is minimized. The working-set strategy, which attempts to have present in main memory every page "needed" by each running process, which balks at the idea of displacing any page in a working set from main memory, is offered as a viable solution to the memory management problem. Choice of the working set parameter τ will depend on compromises among page traffic, wasted memory space, and required availability of pages. The working set size, a convenient quantity to measure, provides a measure of memory demand. Processor demand can be defined to be a prediction of processor time required before an interaction. A balance policy strives to balance demands against equipment by judiciously selecting jobs to run. The notions "demand" and "balance" can play important roles in understanding the complex interactions among computer system components.

Looking at this paper from a slightly different point of view, we have seen four contenders for paging policies: Random, Cyclic, Oldest-unused, and Working-set. For modular programs, the type ultimately expected to predominate in a multiprogrammed environment, Random brings on the highest page traffic, Working-set the lowest. Although Random and Cyclic are inexpensively implemented, the added cos of Working-set is more than offset by its accuracy and compatibility with generalized allocation functions.

ACKNOWLEDGEMENT

The author wishes to thank Jack B. Dennis for many helpful criticisms.

REFERENCES

1. C.V.Ramamoorthy. "The Analytic Design of a Dynamic Look Ahead and Program Segmenting System for Multiprogrammed Computers." Proc. 21 Nat'l Conf. ACM (1966).

2. R.M.Fano and E.E.David. "On the Social Implications of Accessible Computing." AFIPS Conf. Proc. 27 (Nov 1965), 243-247.

3. L.L.Selwyn. "The Information Utility." Industrial Management Review 7, 2, Spring 1966.

4. D.Parkhill. The Challenge of the Computer Utility. Addison-Wesley, 1966.

5. J.B.Dennis. "Segmentation and the Design of Multiprogrammed Computer Systems." JACM 12,4 (Oct 1965), 589-602.

6. J.B.Dennis and E.C.Van Horn. "Programming Semantics for Multiprogrammed Computations." Comm ACM 9 (March 1966), 143-155.

7. B.W.Arden, et al. "Program and Address Structure in a Time-Sharing Environment." JACM 13, 1 (Jan 1966), 1-16.

8. J.H.Saltzer. "Traffic Control in a Multiplexed Computer System." M.I.T. Project MAC technical report MAC-TR-30, July 1966.

9. G.H.Fine, P.V.McIssac, C.W.Jackson. "Dynamic Program Behavior under Paging." Proc. 21 Nat'l Conf. ACM (1966).

10. T.Kilburn, et al. "One-level Storage System." IRE Trans. on Elec. Comp. EC-11, 2 (April 1962).

11. L.A.Belady. "A Study of Replacement Algorithms for a Virtual Storage Computer." IBM Systems Journal 5, 2 (1966), 78-101.

12. Progress Report III, M.I.T. Project MAC (1965-1966), 63-66.

13. P.J.Denning. "Memory Allocation in Multiprogrammed Computers." M.I.T. Project MAC Computation Structures Group Memo 24, March 1966.

14. J.B.Dennis. "Program Structure in a Multi-Access Computer." M.I.T. Project MAC technical report MAC-TR-11.

15. D.W.Fife. "An Optimization Model for Time-Sharing." AFIPS Conf. Proc. 28 (April 1966), 97-104.

APPENDIX -- HARDWARE IMPLEMENTATION OF MEMORY MANAGEMENT

Just as hardware is used to streamline the address-mapping mechanism, so too hardware can be used to streamline memory management. The hardware described here associates a timer with each physical page of main memory a timer to measure multiples of the working set parameter τ.

Each process, upon creation, is assigned an identification number, i, which is used to index the process table. The i-th entry in the process table contains information about the i-th process, including its current demand (p_i, m_i). Because this demand information is stored in a common place, the memory hardware can update the memory demand m_i without calling the supervosor. Whenever a page fault occurs, the new page is located in auxiliary memory and transferred to main memory; then a signal is sent to the management hardware to free a page of main memory in readiness for the next page fault. The hardware selects a page not in any working set and dispatches it directly to auxiliary memory, without bothering the supervisor. This hardware modifies the page table entry pointing to the newly deleted page, turning the "in-core" bit OFF and leaving a pointer to help locate the page in auxiliary memory.

Figure A indicates that with each page of memory there is associated a page register, having three fields:

1. π-field. π is a pointer to the memory location of the page table entry pointing to this page. A page table cannot be moved or removed without modifying π.
2. t-field. t is a timer to measure off the interval τ. The value of τ to be used is found in the t-register. The supervisor modifies the contents of the t-register as discussed below.
3. A-field. A is an "alarm" bit, set to 1 if the timer t runs out.

Operation proceeds as follows:

1. When a page is loaded into main memory, π is set to point to the correct page table entry. The "in-core" bit of that entry is turned ON.
2. Each time a reference to some location within a page occurs, its page register is modified: $\tau \rightarrow t$ and $0 \rightarrow A$. The timer t begins to run down (in real time), taking τ seconds to do so.
3. If t runs down, $1 \rightarrow A$. Whenever a fresh page of memory is needed, the supervisor sends a signal to additional memory hardware (not shown) which scans pages looking for a page with A=1. Such a page is dispatched directly to auxiliary memory. π is used to find the page table entry, turn the "in-core" bit OFF, and leave information there to permit future retrieval of the page from auxiliary memory. Note that a page need not be removed when A=1; it is only subject to removal. This means a page may leave and later re-enter a working set without actually leaving main memory.

The timers t are running down in real time. The value in the t-register must be modifiable by the supervisor for the following reason. As in Figure 6, the running list is cyclic, except now we suppose that each process is given a burst β of processor time (β need not be related to the sampling interval σ), and continues to receive bursts β until its running-list quantum is exhausted. If, on a particular cycle there are n entries in the running list and N processors in service, a given process will be unable to reference any of its pages for about $n\beta/N$ seconds, the time to complete a cycle through the queue. So the supervisor should be able to set τ to some multiple of $n\beta/N$, for otherwise management hardware will begin removing pages of working sets of running processes. However τ should never be less than some multiple of the traverse time T (Figure 1) for otherwise when a process interrupts for a page fault its working set may disappear from core memory.

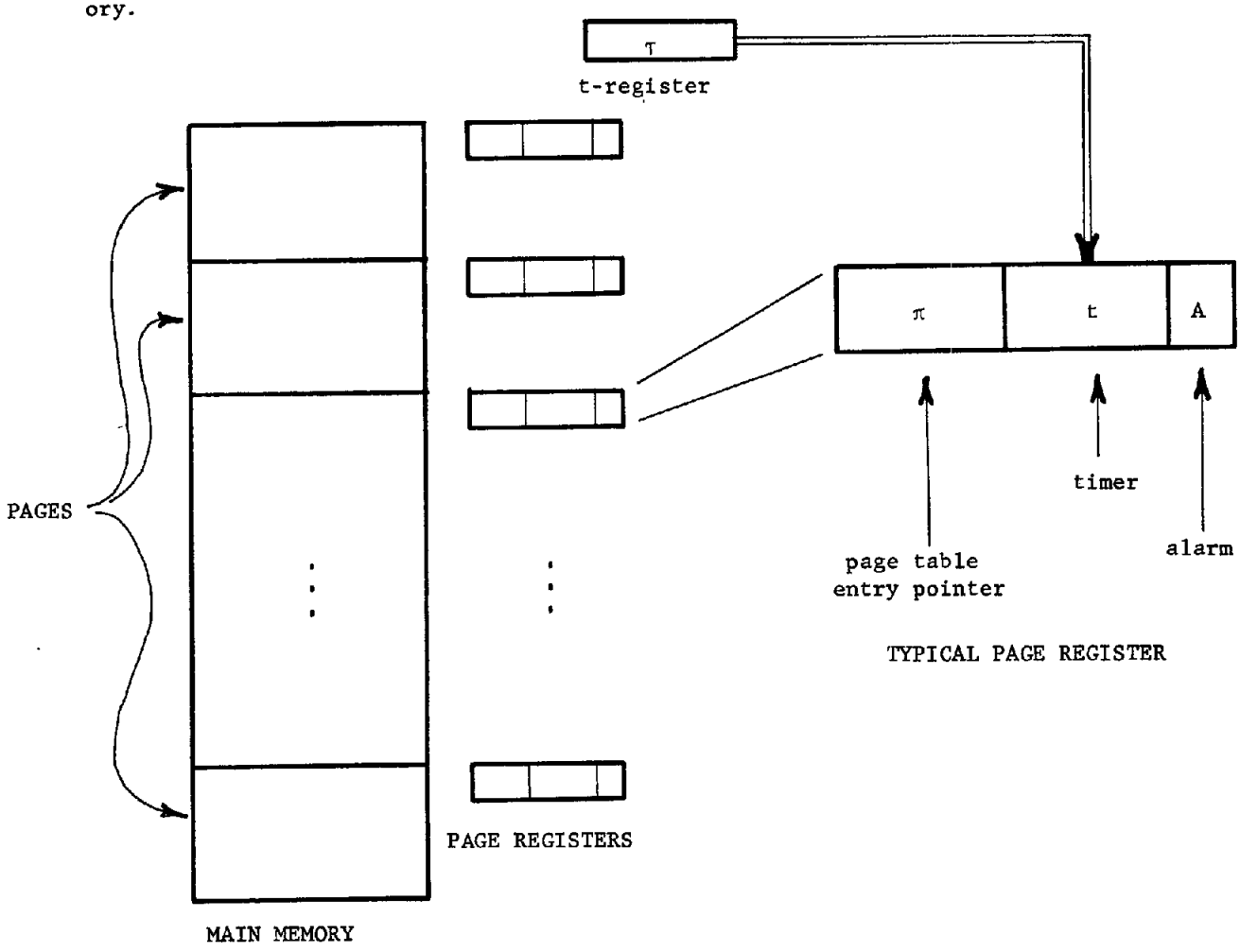

FIGURE A. Memory Management Hardware.